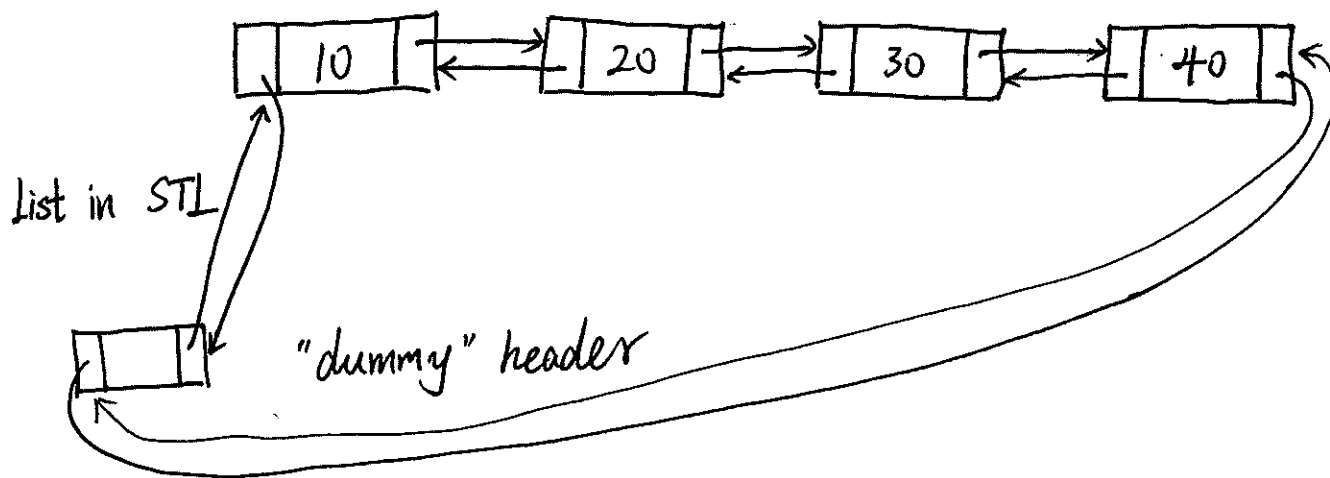


## Container: List

Review:

A doubly linked list is a collection of nodes. Each node contains three fields:

1. an information field holds the actual element on the list
2. a next address field contains the address of the next node
3. a previous address field contains the address of the previous node



## Implementing List

\* A class representing the ListNode

1. data members / member variables

- data

T

- next

ListNode\*

- prev

ListNode\*

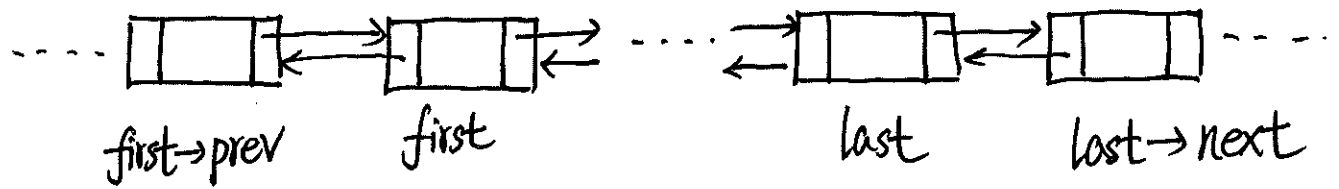
## 2. member functions of ListNode

- unhooks the range [first, last] from a list

e.g.  $1 \rightleftharpoons [2 \rightleftharpoons 3] \rightleftharpoons 4 \rightleftharpoons 5$

$1 \rightleftharpoons 4 \rightleftharpoons 5$

static void unhook\_range(ListNode\* first, ListNode\* last);

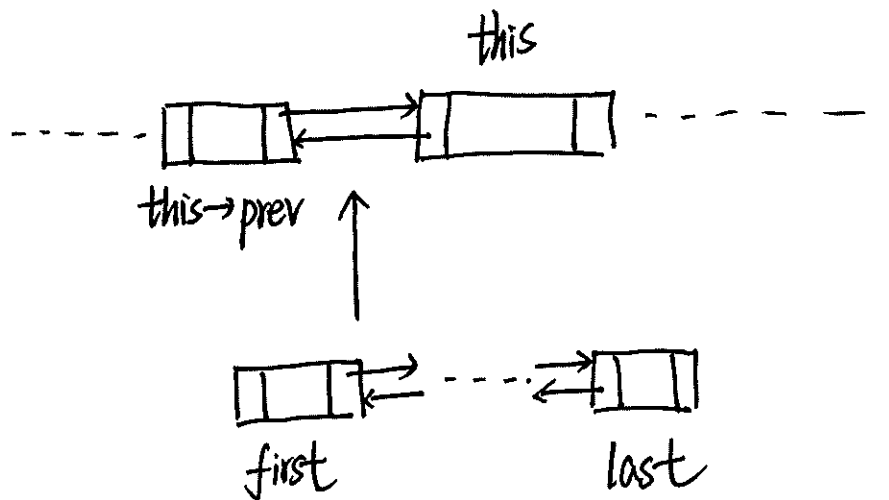


$first \rightarrow prev \rightarrow next = last \rightarrow next;$

$last \rightarrow next \rightarrow prev = first \rightarrow prev;$

- hooks the range [first, last] before this

void hook\_range(ListNode\* first, ListNode\* last);



this → prev → next = first;

first → prev = this → prev;

this → prev = last;

(order is important)

last → next = this;

- insert one node before this

```
void hook(ListNode* node) {  
    hook_range(node, node);  
}
```

- unhook the current node (this) from a list

```
void unhook() {  
    ListNode::unhook_range(this, this);  
}
```

\* A struct representing a list iterator: ListIterator

The difference between struct and class is the default accessibility of member variables and functions.

For struct, it's public.

For class, it's private.

1. data member(s):

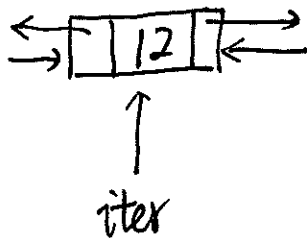
- m\_nodePtr      ListNode\*

2. member functions

- constructor

- dereference operator (operator\*)

e.g.



what is \*iter?

ListNode { data  
prev  
next

Td operator\*() const {

return m\_nodePtr -> data;

}



## Function Overloading

pre-increment:

```
ListIterator& operator++() {  
    m_nodePtr = m_nodePtr → next;  
    return *this;  
}
```

post-increment:

```
ListIterator operator++(int) {  
    // create a ListIterator which is a copy of *this  
    // move m_nodePtr to its next  
    // return the copy we created  
}
```

- Decrement Operators (operator--)

pre-decrement:

```
ListIterator& operator--();
```

post-decrement:

```
ListIterator operator--(int);
```

Leave the rest to the readers.

\* A struct representing a list const-iterator: ListConstIterator

iterator vs const-iterator

e.g. In struct ListIterator:

```
T& operator*() const;
```

In struct ListConstIterator:

```
const T& operator*() const;
```

\* A class List

## 1. Data Members

- m\_header      ListNode
- m\_size      size\_t

## 2. Member Functions

- Constructors & Destructor

- Iterator accessors

begin()

end()

- Data accessors

front()

back()



```
ListIterator begin() {
```

```
    ListIterator iter(m_header.next);
```

```
    return iter;
```

```
}
```

```
ListIterator begin() {
```

```
    return {m_header.next};
```

```
}
```

```
ListIterator end() {
```

// Which node is end() pointing to ? m\_header

// Can we return {m\_header} ? No

m\_header is a node.

the constructor needs a node\*

e.g. m\_header.next

```
(return {&m_header};)
```

```
}
```

```
T& front() {  
    return *begin();  
}
```

```
T& back() {  
    return * (--end());  
}
```

↑ can NOT be end() --

insert (pos, value)

Hint: hook() is to hook a node before this

① What is the parameter of hook() ?

- A listNode\*

② Which data type/object can call hook() ?

- ListNode / ListNode\*

step1 // create a new ListNode\* we need to insert  
└ when we use "new", we are creating a pointer.

step2 // call hook(). Can pos call hook() ? No

// increase m\_size

// return an iterator

e.g.

new Node(value)

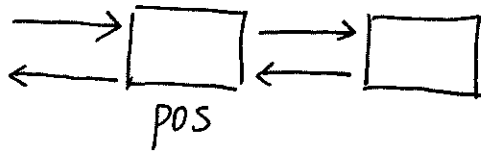
erase

- ① We need to call `unhook()`, but `pos` which is an iterator cannot call `unhook()`.

What to do? Hint: A `ListNode` or `ListNode*` can call `unhook()`

- ② We should return an iterator referring to the next node.

But

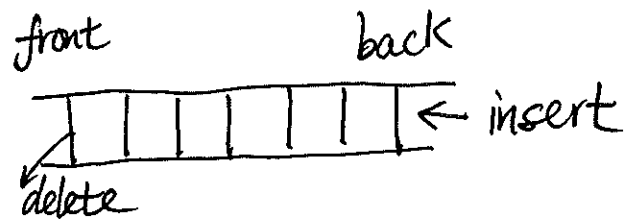


if we first `unhook pos.m_nodePtr`, then we can no longer get access to the next node.

Please solve these!

## Stack & Queue

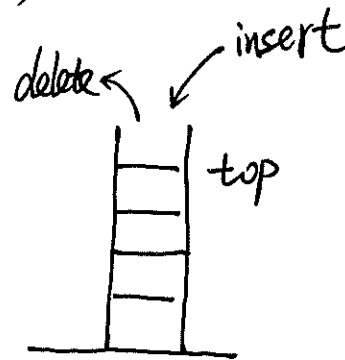
Queue: (FIFO principle)



A queue is a sequence of objects, where element access and deletion are restricted to the first element in the queue, which is called the front.

Element insertion is restricted to the end of the sequence, which is called the back (rear).

Stack: (LIFO principle)



Element access, deletion

and insertion are all restricted to the top.

In C++, they are implemented as container adaptors, which use a container class as its underlying container, providing a specific set of functions.

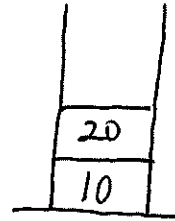
underlying containers: vector, list, ...

check [cplusplus.com](http://cplusplus.com)

member functions :

stack: top, pop, push, empty, size, etc.

e.g. `std::stack<int> mystack;`  
`mystack.push(10);`  
`mystack.push(20);`  
`mystack.top() = mystack.top() - 5;`



Queue: front, pop, push, empty, size, etc.

e.g. `std::queue<int> myQ;`  
`myQ.push(77);`  
`myQ.push(10);`  
`myQ.front() = myQ.front() - 7;`

